Amazon Mentorship Program
Design & Planning Document

Project Title: Flora — A Virtual Garden Companion for Lebanon

Team Members:

- Omar Aghbari
- Abduallah Al-Qubli
- Mohammad Al-Haiqi
- Abdulaziz Al-Shujaa
- Ranim Khattar

Mentor: Ahmad Dimachkie

Domain: Sustainability · Green Technology

1. Introduction

Problem Statement

Lebanese hobbyist gardeners — from apartment balconies in Beirut to mountain homes and suburban backyards — face a fragmented and unsupported experience. When a plant shows signs of disease, there is no reliable, locally-adapted tool to help diagnose it. The problem runs deeper than diagnosis: gardeners have no centralized way to track the plants they own, remember watering schedules, monitor plant health over time, or receive seasonal care reminders suited to Lebanon's specific climate zones.

Existing global apps such as PictureThis, Planta, and Plantix are not adapted to Lebanese plants, climate, or language, and none of them connect gardeners to one another or to the local gardening economy of nurseries and agricultural experts.

Project Objective

Flora is a bilingual (Arabic and English) mobile application for iOS and Android that acts as a complete virtual garden companion. Onboarding is deliberately frictionless: a new user signs up with just a username and password — no email, since this is a simple consumer app — and lands directly on their home screen, which is their virtual garden. From there they add the plants they own, either by name (searching a plant catalog) or by photo (image identification), organized by space — balcony, backyard, indoor, or rooftop. The app then becomes a living dashboard for the garden's health and schedule. Flora delivers this through three integrated capabilities:

- **Diagnosis & image checking —** photograph a plant to identify it or assess its health, with clear, localized care guidance.
- **Virtual garden —** track each plant by space, with automated watering and season-aware reminders tuned to Lebanon.
- **Community —** a shared space where gardeners exchange knowledge, with a future B2B layer linking them to local nurseries and experts.

Flora ships as a consumer mobile product on a freemium model, Lebanon-first by design and bilingual throughout.

2. Key Features

Feature 1 — Diagnosis & Image Checking

The user captures or uploads a photo of a plant. Flora uses it in two ways: to identify the plant when adding it to a garden, and to assess its health and flag likely diseases.

- **Upload:** the app requests a pre-signed URL and uploads the image directly to Amazon S3, keeping large files off the API path.
- **Recognition:** the upload triggers an AWS Lambda worker that runs recognition. For launch this calls a third-party plant-ID API; the pipeline is designed so we can later self-host the model on AWS (Amazon Rekognition Custom Labels or SageMaker) without changing the app.
- **Care guidance:** results are matched to structured care information from the plant catalog and returned in the user's language. (An LLM-generated advice layer via Amazon Bedrock is deferred to a later phase.)
- **History:** each result is stored against the specific plant so the user can see how its health evolves over time; low-confidence results are flagged and can be escalated to the community.

Feature 2 — Virtual Garden: Tracking & Watering

The virtual garden is the heart of the product: a structured, always-current model of everything the user is growing, and the engine that keeps them on schedule.

- **Adding plants:** users add a plant by name (searching the plant catalog) or by photo (image identification), and organize plants by space — balcony, backyard, indoor, rooftop.
- **Scheduling:** per-plant watering schedules and seasonal care reminders are tuned to Lebanon's climate zones.
- **Reminders:** Amazon EventBridge Scheduler fires due reminders to an AWS Lambda function, which dispatches push notifications through Amazon SNS.
- **Growth tracking:** users log periodic photos and notes, producing a visual health timeline for each plant.

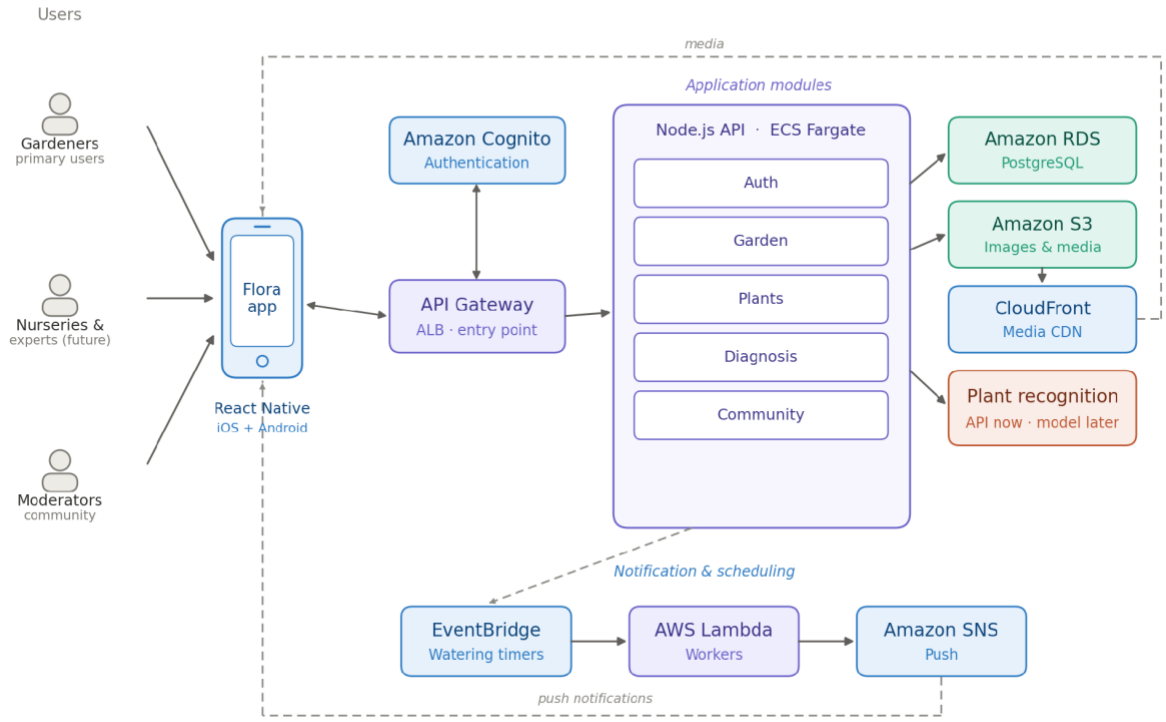
Feature 3 — Community

Community is what makes Flora Lebanon-first rather than a generic plant utility: a place for local gardeners to help each other and, over time, to connect with the local gardening economy.

- **Feed:** users post photos, questions, and tips, and can comment on, like, and follow one another.
- **Local knowledge:** advice is grounded in Lebanese plants, seasons, and conditions rather than generic global guidance.
- **Media delivery:** post images are stored in Amazon S3 and served through Amazon CloudFront for fast loading on mobile; uploads can be screened with Amazon Rekognition moderation labels.
- **Future B2B:** a later layer connects users to nurseries and agricultural experts, opening a revenue path beyond the consumer freemium tier.

3. Low-Level Design

Flora runs as a modular monolith: a single Node.js/Express service on Amazon ECS (Fargate), organized internally into feature modules — auth, garden, plants, diagnosis, and community — each owning its own routes, service logic, and validators over a shared Prisma client and a common ApiResponse convention. This gives the five-person team clean boundaries and shared types now, without the networking and deployment overhead of separately deployed microservices; any module can be split out into its own service later if it genuinely needs to scale on its own. A React Native client talks to the API behind Amazon Cognito. The diagram below shows how a request flows through the system.



Reading the diagram: it flows left to right. Flora's users — gardeners today, nurseries and experts later, plus community moderators — reach the app through the React Native client, which calls the Node.js API behind Amazon Cognito. Inside the API, the feature modules (Auth, Garden, Plants, Diagnosis, Community) handle requests and talk to the data layer: PostgreSQL on RDS, media in S3 delivered through CloudFront, and plant recognition (an external API now, an AWS-hosted model later). The band along the bottom is the scheduling path — EventBridge triggers Lambda, which sends watering reminders to users as push notifications through Amazon SNS. The two dashed lines are the paths users feel directly: media delivery and push notifications.

4. Technologies

A. Programming Languages

- **JavaScript / TypeScript (Node.js) —** one language across the mobile client (React Native) and the backend (Express), so a five-person team shares types, validators, and helpers across the whole stack. TypeScript adds compile-time safety.

- **SQL (PostgreSQL dialect) —** for Flora's strongly relational data (gardens, plants, logs, and the community graph).

B. Frameworks & Libraries

- **React Native (with Expo) —** one codebase produces both the iOS and Android apps, with first-class internationalization and right-to-left support for Arabic.
- **Node.js + Express —** a lightweight REST API, structured as feature modules the whole team can contribute to.
- **Prisma ORM —** type-safe database access and versioned migrations over PostgreSQL, matching the team's experience.
- **i18next —** manages Arabic/English translations throughout the app.

C. AWS Services (at a glance)

- **Cognito —** username + password authentication.
- **ECS Fargate + Application Load Balancer —** runs the containerized Node.js API without managing servers.
- **RDS (PostgreSQL) —** primary relational datastore; Aurora Serverless v2 is the scale-up path.
- **S3 + CloudFront —** media storage and fast content delivery.
- **Lambda —** event-driven workers for recognition and reminder dispatch.
- **EventBridge Scheduler —** per-plant watering and seasonal reminders.
- **Rekognition Custom Labels / SageMaker —** the AWS-hosted option for plant recognition and disease detection.
- **SNS —** mobile push notifications.
- **CloudWatch & SES —** logging/monitoring and transactional email.

5. Design Decisions & Alternatives

Every layer of Flora runs on AWS. The one exception is the plant-recognition and plant-catalog data source: AWS does not provide a plant species database, so we start on a third-party plant API and keep a documented path to self-hosting the model on AWS (Rekognition Custom Labels or SageMaker) for a 100%-AWS stack. The table below records why each service was chosen and the main alternatives we weighed.

Decision	What we chose & why	Alternatives considered (✓ pro / ✗ con)
ComputeECS (Fargate)	Run one containerized Node.js API with no servers to manage; it scales on demand and keeps the modular codebase in a single, simple deployment.	**Lambda (serverless):** ✓ scales to zero, cheap when idle ✗ cold starts, needs RDS Proxy, awkward local dev **EC2:** ✓ full control, cheap at steady load ✗ you patch, scale and secure the box **Full microservices:** ✓ independent scaling per service ✗ networking + ops overhead, overkill for a 5-person app

Decision	What we chose & why	Alternatives considered (✓ pro / ✗ con)
AuthenticationCognito	Managed, secure username + password sign-in — no email required. Password storage and future MFA/social login are handled for us, and it stays AWS-native.	**Self-managed JWT:** ✓ dead simple, no extra service ✗ we own hashing, breaches and token revocation **Auth0 / Firebase:** ✓ fast to set up ✗ not AWS, adds an external dependency
DatabaseRDS (PostgreSQL)	Flora's data is relational (users → gardens → plants → logs; posts → comments → likes). Prisma over Postgres matches the team's experience and the instance is free-tier eligible.	**DynamoDB:** ✓ serverless, effortless scaling ✗ rigid access patterns, joins and feeds are painful **Aurora Serverless v2:** ✓ auto-scales, low idle cost ✗ min-capacity cost above free tier **Postgres on EC2:** ✓ cheapest, full control ✗ you run backups, patching, failover
Storage + deliveryS3 + CloudFront	Durable, cheap image storage with edge caching so photos load fast on mobile. Pre-signed uploads keep large files off the API path.	**S3 only (no CDN):** ✓ simpler and cheaper ✗ slower for distant users, no caching **Amazon EFS:** ✓ shared file system ✗ wrong fit for user media + web delivery
Plant recognition & diagnosisExternal API now → AWS model later	Start on a third-party plant-ID API (e.g. Plant.id, Pl@ntNet) to ship fast with broad species coverage and no training. The pipeline is built so the model can move in-house on AWS once we have labelled data.	**Rekognition Custom Labels:** ✓ AWS-native, managed AutoML, little ML skill needed ✗ needs a labelled dataset, endpoint billed hourly while running **SageMaker:** ✓ full control, host any model (e.g. PlantVillage CNN) ✗ the most ML and ops effort **3rd-party API:** ✓ no training, fast, wide coverage ✗ per-call cost/limits, data leaves AWS
Plant catalog & care dataPlant API cached in RDS	Species taxonomy and care instructions aren't something AWS provides, so we pull them from a plant-data API and cache what we use in Postgres. This supplies care guidance without an LLM for now.	**Curate our own dataset:** ✓ Lebanon-tailored, no external dependency ✗ large manual data-collection effort **Bedrock (LLM advice):** ✓ rich, conversational guidance ✗ deferred — no chatbot in this phase
Scheduled remindersEventBridge Scheduler	Managed, reliable per-plant watering schedules with built-in retries and no always-on cron process to babysit.	**Cron in the container:** ✓ trivial to add ✗ tied to one instance, no managed retries **Step Functions:** ✓ great for multi-step workflows ✗ overkill for simple reminders
Push notificationsAmazon SNS	Simple, cheap mobile push for watering reminders and community activity — enough for v1.	**Amazon Pinpoint:** ✓ campaigns, segmentation, analytics ✗ heavier than we need now **In-app only:** ✓ nothing to run ✗ users miss time-sensitive reminders
Monitoring & emailCloudWatch + SES	CloudWatch centralises logs, metrics and alarms; SES covers any transactional email (e.g. account recovery) if we add it later.	**3rd-party (Datadog, etc.):** ✓ richer dashboards ✗ extra cost, not AWS